Static Website Hosting on AWS

Proposed Solution

I would host the site in a **private Amazon S3 bucket** and serve it through **Amazon CloudFront**, with a free **AWS Certificate Manager (ACM)** certificate for HTTPS and **Route 53** alias records pointing the domain at CloudFront.

This has the least management overhead because there are no servers to patch, no certificates to renew manually and nothing to scale. It is also the most cost-effective option at this size. 500 MB of storage costs about $0.01 per month, and CloudFront's always-free tier covers 1 TB of data transfer and 10 million requests per month.

Assumptions

- The site is fully static (HTML, CSS, JavaScript and images) with no server-side code.
- Traffic is moderate and stays within or close to the CloudFront free tier.
- The site is served on the apex domain (example.com), and www redirects to it.

Why Not Other Options

- **S3 static website endpoint alone:** It does not support HTTPS on a custom domain, and it requires a public bucket.
- **AWS Amplify Hosting:** It is easy to set up, but it costs more per GB served and gives less control over how the origin is secured.
- **EC2 or Lightsail:** Too much overhead (patching, scaling and certificates) for 10 static pages.

Keeping Content Private from Direct Access

- **S3 Block Public Access** is turned on, so the bucket and its objects can never be made public.
- **Origin Access Control** lets CloudFront sign its requests to S3. I chose OAC over the older Origin Access Identity because AWS recommends it.
- **The bucket policy** allows s3:GetObject only for the CloudFront service, and only when the request comes from my specific distribution (an AWS:SourceArn condition).
- **HTTPS only:** CloudFront redirects HTTP to HTTPS and enforces TLS 1.2 or higher.

The result: requests through the domain return the page, while direct requests to the S3 URL return **403 Access Denied**.

Deploying Updates

I would use a CI/CD pipeline (GitHub Actions):

1. A push to the main branch triggers the pipeline.
2. The pipeline authenticates to AWS with the access keys of a **dedicated IAM user**. The keys are stored as encrypted **GitHub Secrets** (AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY) and are never committed to the repository.
3. It runs aws s3 sync --delete to upload changed files and remove deleted ones.
4. It runs a CloudFront **invalidation** (/*) so visitors get the new content right away. The first 1,000 invalidation paths each month are free.

To roll back, I would revert the commit and let the pipeline redeploy. I would also enable S3 versioning as a safety net.